

Capstone Project — Certificate Program in Artificial Intelligence and Machine Learning

Zepto Data & AI Platform

An end-to-end AI/ML engineer is expected to move comfortably across the full stack: pulling raw data from the wild, cleaning and storing it properly, understanding it visually, building and evaluating predictive models on it, and — increasingly — wrapping intelligence around it with a deployed GenAI service. In this capstone you are joining Zepto's analytics guild as an incoming AI/ML engineer, and your first assignment is to ship **one connected platform** made of three internally-linked capabilities: a data-engineering pipeline that turns raw scraped data into a clean relational store, an analytics pipeline that profiles and models a customer-style dataset end to end, and a GenAI support assistant that answers policy questions grounded in Zepto's own documents. All three live together in **one repository** — this is a single, coherent submission, not three unrelated exercises. Every technique required below is a standard, well-documented technique in professional AI/ML practice, with widely available tooling and documentation support.

You may build the three modules in any order, but they are meant to read as one story: a data pipeline feeds Zepto's analysts clean structured data (`/data_pipeline`), an analytics pipeline shows how Zepto would profile and predict customer/passenger-style outcomes end to end (`/analytics`), and a support assistant shows how Zepto would put a grounded GenAI service in front of its own policies (`/support_assistant`). Each module is graded independently against its own criteria, but they all live in, and are submitted as, **one single public GitHub repository**.

Total marks: 100 · **Modules:** 3 (`/data_pipeline` — 25, `/analytics` — 50, `/support_assistant` — 25)

Submission Guidelines (read first)

- Submit **exactly ONE public GitHub repository link** for this entire project. Do **not** create separate repositories per module — there is one repo, containing all three module folders (`/data_pipeline` , `/analytics` , `/support_assistant`) at its root, plus one root `README.md` .
- The root `README.md` must document: how to set up the project (a `requirements.txt` per module, or one consolidated `requirements.txt` , your choice — state which), how

to run each of the three modules end to end, and a short summary of your design decisions for each module. Any written interpretation a task asks for lives as Markdown text — inside the root README, a module-level README, or notebook Markdown cells — not in a separate document type.

- All deliverables are **textual**: code lives in the repository as `.py` / `.ipynb` files, and every write-up lives as Markdown text inside the repository. **No screenshots, images, PDFs, slide decks, presentations, video, or audio are required or accepted anywhere in this project.** Chart image files your code generates (e.g., saved `.png` plots) may live inside the repository as supporting artifacts, but they are never a substitute for the required written interpretation — a grader must be able to assess your work from the text alone.
- **Academic integrity**: the code, analysis, and written interpretations must be your own. You may use standard library/framework documentation, but the reasoning and implementation must be authored by you.
- **Deadline**: submit your single repository link by the date/time communicated separately on the LMS for this evaluation.
- **Git workflow (scored once, against the whole repository)**. Your repository's commit history should show at least one feature branch that was created, committed to at least twice, and then merged back into `main` (visible via `git log --graph --all` or the repository's branch/merge-commit history). This is expected practice for the project as a whole and is checked once against the entire repo's history — it is not required separately inside each module folder, and it is not double-counted across modules. It is a scored criterion under `/data_pipeline`'s rubric (see `grader_rubric.md`), but the branch and merge activity itself may touch any part of the repository.
- **No paid services required anywhere in this project.** Every external service used below has an explicitly free path stated in its module — read each module's fallback note before you begin, since a fallback stated in one module does not carry over to another module's separate dependency.